



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	BALAJIAH GUDIPUDI
Student Name	PALAVALLASA RAHUL
Roll Number	2520090016
Branch	CSIT
Course Outcome	COURSE OUTCOME - 5

Case Study Topic: Cricket Scoring Service using Counting Sort and Radix Sort

Work Done Summary

In this case study, we analyzed the sorting of cricket delivery records using composite keys (Over, Ball). Since the key ranges are small (Over = 0–49 and Ball = 1–12), counting sort and radix sort become highly efficient alternatives to comparison-based algorithms.

The implementation demonstrates stable counting sort, radix-style sorting using multiple passes, algorithm comparison, and external sorting techniques for large archives.

Implementation Details

1. Represented each delivery as (Over, Ball).
2. Applied Counting Sort on Ball number.
3. Applied Stable Counting Sort on Over number.
4. Preserved ordering using stability.
5. Compared Merge Sort, Quick Sort, Heap Sort, Counting Sort, and Radix Sort.
6. Discussed external merge sort for large datasets.

(a) Sorting Using LSD Radix Style

Input

(2, 4)
(1, 1)
(3, 6)
(1, 5)
(2, 2)
(3, 1)
(1, 3)
(2, 6)
(3, 4)
(1, 2)

Pass 1: Stable Counting Sort by Ball Number

Ball = 1 Bucket

(1, 1)
(3, 1)

Ball = 2 Bucket

(2, 2)
(1, 2)

Ball = 3 Bucket

(1, 3)

Ball = 4 Bucket

(2, 4)

(3, 4)

Ball = 5 Bucket

(1, 5)

Ball = 6 Bucket

(3, 6)

(2, 6)

Result After Pass 1

(1, 1)

(3, 1)

(2, 2)

(1, 2)

(1, 3)

(2, 4)

(3, 4)

(1, 5)

(3, 6)

(2, 6)

Pass 2: Stable Counting Sort by Over Number

Over = 1 Bucket

(1, 1)

(1, 2)

(1, 3)

(1, 5)

Over = 2 Bucket

(2, 2)

(2, 4)

(2, 6)

Over = 3 Bucket

(3, 1)

(3, 4)

(3, 6)

Final Sorted Order

(1, 1)

(1, 2)

(1, 3)

(1, 5)

(2, 2)

(2, 4)

(2, 6)

(3, 1)

(3, 4)

(3, 6)

Why Stability is Important

Stability ensures that when sorting by Over in the second pass, deliveries having the same Over retain the Ball ordering established in the first pass.

Without stability:

(1, 1)

(1, 3)

(1, 2)

could appear incorrectly.

Thus LSD Radix Sort relies on stable counting sort.

(b) Java Program (Set1CO5.java)

Output:

(1,1)

(1,5)

(2,4)

(3,6)

Process finished

Time Complexity

Counting Frequency

$O(n)$

Prefix Sum

$O(K)$

Output Construction

$O(n)$

Overall:

$O(n + K)$

Why Reverse Iteration?

When elements with equal keys appear:

A B C

reverse traversal ensures:

A B C

remains unchanged after sorting.

Thus stability is preserved.

Working Memory

$\text{count}[] = O(K)$

$\text{output}[] = O(n)$

Total:

$O(n + K)$

(c) Comparison of Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable	In-place
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	Yes	No
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	Yes	No

Expected Wall Clock for $n = 1,000,000$

Algorithm	Approx Runtime
Merge Sort	0.8–1.5 sec
Quick Sort	0.5–1 sec
Heap Sort	1–2 sec
Counting Sort	< 0.1 sec
Radix Sort	< 0.15 sec

Winner

For this workload:

Over Range = 50
Ball Range = 12

Counting Sort and Radix Sort dominate.

Reason:

Small key range
Huge dataset
Linear-time sorting

(d) External Sorting (30 GB Archive)

Problem

Archive Size:

30 GB

Available RAM:

16 GB

Data cannot fit into memory.

Phase 1: Chunk and Sort

Split archive into chunks.

Chunk 1 → 15 GB
Chunk 2 → 15 GB

Each chunk:

Load into RAM
Sort internally
Write back to disk

Produces:

Sorted Run 1
Sorted Run 2

Phase 2: K-Way Merge

Use a Min Heap.

Heap contains:

smallest element from each run

Repeatedly:

Extract minimum
Write to output file
Insert next element from same run

Until all runs are exhausted.

I/O Cost

External Merge Sort:

$O(N \log(N/M))$

where:

N = file size
M = RAM size

Given:

N = 30 GB
M = 16 GB

Number of runs:

$30 / 16$

≈ 2 runs

Merge passes:

$\log_2(2)$

= 1

Total cost roughly:

Read 30 GB
Write 30 GB

Read 30 GB
Write 30 GB

≈

120 GB disk I/O

Result

The composite key sorting was successfully performed using stable counting sort. The final order obtained was:

(1, 1)
(1, 2)
(1, 3)
(1, 5)
(2, 2)
(2, 4)
(2, 6)
(3, 1)
(3, 4)
(3, 6)

Counting Sort and Radix Sort outperform comparison-based algorithms because the key ranges are extremely small.

Time Complexity Summary

Algorithm	Complexity
Counting Sort	$O(n+k)$
Radix Sort	$O(d(n+k))$
Merge Sort	$O(n \log n)$
Quick Sort	$O(n \log n)$ Avg
Heap Sort	$O(n \log n)$

Conclusion

For cricket delivery records, Counting Sort and Radix Sort are the most suitable choices due to their linear-time complexity and stable behavior. They efficiently handle millions of records while preserving the correct chronological ordering of deliveries. When datasets exceed RAM capacity, External Merge Sort provides a scalable disk-based solution for processing multi-gigabyte archives.

GitHub Link: <https://github.com/palavalasarahul-blip/DSA-CASE-STUDIES.git>

Student Signature	Faculty Signature